

inVision U — Архитектура решения

Содержание

- [Обзор](#обзор)
- [Бизнес-логика и флоу](#бизнес-логика-и-флоу)
- [Стек технологий](#стек-технологий)
- [Архитектура системы](#архитектура-системы)
- [Docker-инфраструктура](#docker-инфраструктура)
- [Роли пользователей и интерфейсы](#роли-пользователей-и-интерфейсы)
- [Схема базы данных](#схема-базы-данных)
- [API-эндпоинты](#api-эндпоинты)
- [Аутентификация и авторизация](#аутентификация-и-авторизация)
- [Frontend-архитектура](#frontend-архитектура)
- [ML-сервис](#ml-сервис)
- [Структура проекта](#структура-проекта)

Обзор

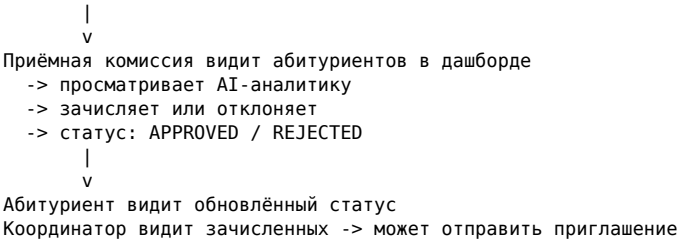
inVision U — AI-система поддержки отбора кандидатов для университета inVision U by inDrive (хакатон Decentrathon 5.0). Система автоматизирует полный цикл приёмной кампании: от создания программы с вопросами до финального зачисления абитуриента приёмной комиссией.

Ключевая идея: абитуриент отвечает на набор вопросов (21 системный + кастомные), AI анализирует ответы, проверяет на AI-генерацию и формирует детальную оценку, координатор отбора фильтрует лучших и отправляет в приёмную комиссию на финальное решение.

Бизнес-логика и флоу

Основной поток

```
Координатор создаёт программу
+ выбирает вопросы из банка (21 системный + свои)
|
v
Абитуриент регистрируется (email + phone + пароль)
-> видит список программ
-> выбирает программу
-> проходит анкету (отвечает на все вопросы)
|
v
AI анализирует ответы (mock / ML-сервис)
-> score/100, program_match%, потенциал роста, growth_path_score
-> сильные стороны, зоны развития, AI-резюме
-> проверка на AI-генерацию (ai_detection_flags)
-> статус: ANALYZED
|
v
Координатор отбора видит результаты AI в разделе "Отчёты"
-> просматривает анализ + AI-флаги
-> открывает Досье (ответы абитуриента + профиль)
-> отправляет лучших в приёмную комиссию
-> статус: SENT_TO_MANAGER
```



Жизненный цикл абитуриента (статусы)

PENDING -> PROCESSING -> ANALYZED -> HR_REVIEW -> SENT_TO_MANAGER -> APPROVED
-> REJECTED

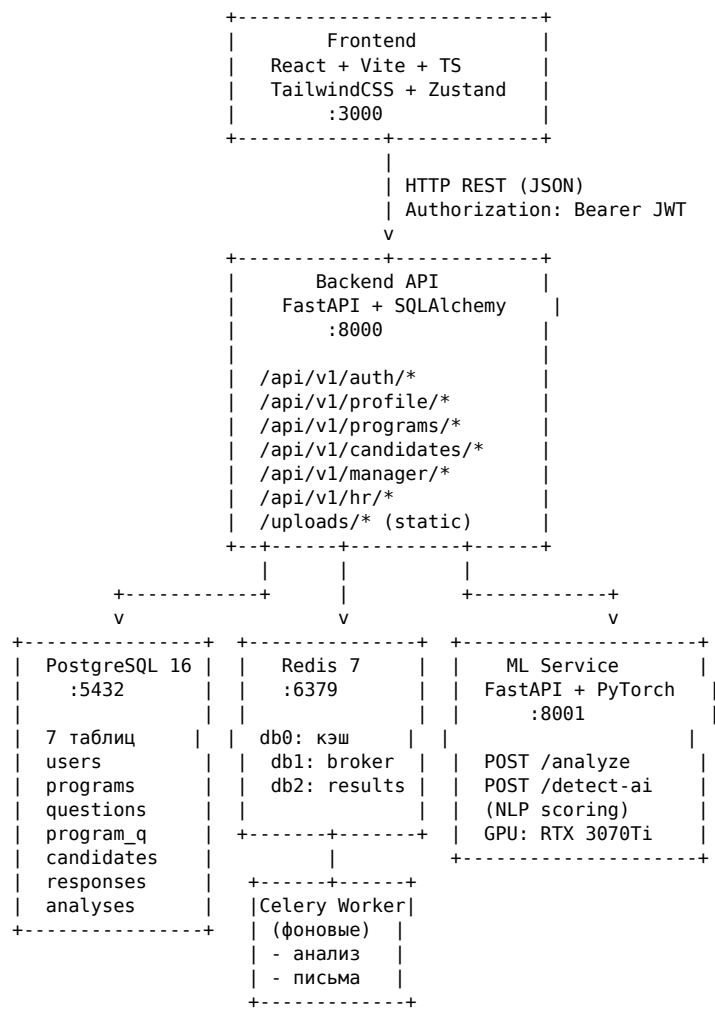
Статус	Кто видит	Описание
PENDING	Система	Ответы получены, анализ не начат
PROCESSING	Система	ML-сервис обрабатывает
ANALYZED	Координатор	Анализ завершён, ждёт ревью координатора
HR_REVIEW	Координатор	Координатор изучает абитуриента
SENT_TO_MANAGER	Комиссия	Координатор отправил на финальное решение
APPROVED	Все	Приёмная комиссия зачислила
REJECTED	Все	Приёмная комиссия отказала

Стек технологий

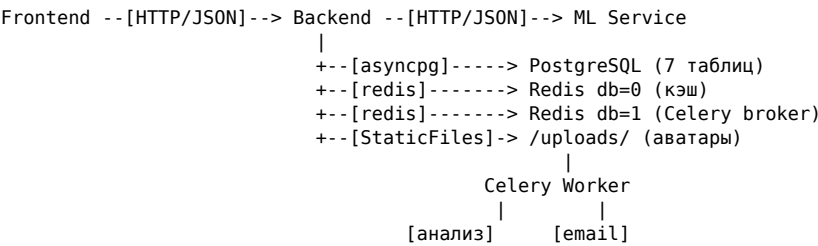
Слой	Технологии
Frontend	React 18, TypeScript, Vite, TailwindCSS, Zustand, React Router v6, Axios
Backend API	Python 3.11+, FastAPI, SQLAlchemy 2.0 (async), Pydantic v2
ML Service	Python 3.11+, FastAPI, PyTorch, Transformers, scikit-learn
База данных	PostgreSQL 16
Кэш / Брокер	Redis 7 (кэш db=0, Celery broker db=1, Celery results db=2)
Фоновые задачи	Celery
Аутентификация	JWT (access 30min + refresh 7d), bcrypt, python-jose
Файлы	StaticFiles (аватары -> /uploads/avatars/)
Контейнеризация	Docker, Docker Compose

Архитектура системы

Диаграмма сервисов



Связи между сервисами



Docker-инфраструктура

Проект запускается через docker-compose.yml (production) + docker-compose.dev.yml (dev c hot-reload).

Сервис	Образ	Порт	Зависимости	Healthcheck
postgres	postgres:16-alpine	5432	—	pg_isready
redis	redis:7-alpine	6379	—	redis-cli ping
backend	./backend/Dockerfile	8000	postgres, redis	—

ml-service	./ml-service/Dockerfile	8001	—	—
celery-worker	./backend/Dockerfile	—	postgres, redis	—
frontend	./frontend/Dockerfile	3000	backend	—

Запуск (dev):

```
docker-compose -f docker-compose.yml -f docker-compose.dev.yml up -d --build
```

Volumes: postgres_data, redis_data

Роли пользователей и интерфейсы

Абитуриент (candidate)

Создаётся автоматически при регистрации. Не может выбрать другую роль.

Страница	Маршрут	Описание
Программы	/programs	Список активных программ
Анкета	/questionnaire/:programId	Пошаговая форма с вопросами
Мой статус	/status	Статус рассмотрения заявки
Профиль	/profile	Редактирование профиля, аватар

Координатор отбора (hr)

Создаётся вручную в БД (UPDATE role = 'HR'). Управляет программами и фильтрует абитуриентов.

Страница	Маршрут	Описание
Программы	/hr	Список программ, создание новых
Создание программы	/hr/programs/create	Форма + выбор вопросов из банка
Детали программы	/hr/programs/:id	Программа + абитуриенты
Ревью абитуриента	/hr/candidates/:id	AI-анализ + AI-флаги + отправка в комиссию
Отчёты	/hr/reports	Программы с количеством абитуриентов
Отчёт по программе	/hr/reports/program/:id	Все абитуриенты по программе (все статусы)
Анализ абитуриента	/hr/reports/candidate/:id	AI-аналитика + кнопка "Досье"
Досье абитуриента	/hr/reports/candidate/:id/dossier	Профиль + AI-оценка + AI-флаги + ВСЕ ответы
Зачисленные	/hr/approved	Абитуриенты после зачисления комиссией
Профиль	/profile	Редактирование своего профиля

Приёмная комиссия (manager)

Создаётся вручную в БД (UPDATE role = 'MANAGER'). Принимает финальное решение о зачислении.

Страница	Маршрут	Описание
Дашборд	/manager	Абитуриенты, отправленные координатором (SENT_TO_MANAGER)
Детали абитуриента	/manager/candidate/:id	AI-аналитика + зачислить/отклонить
Профиль	/profile	Редактирование своего профиля

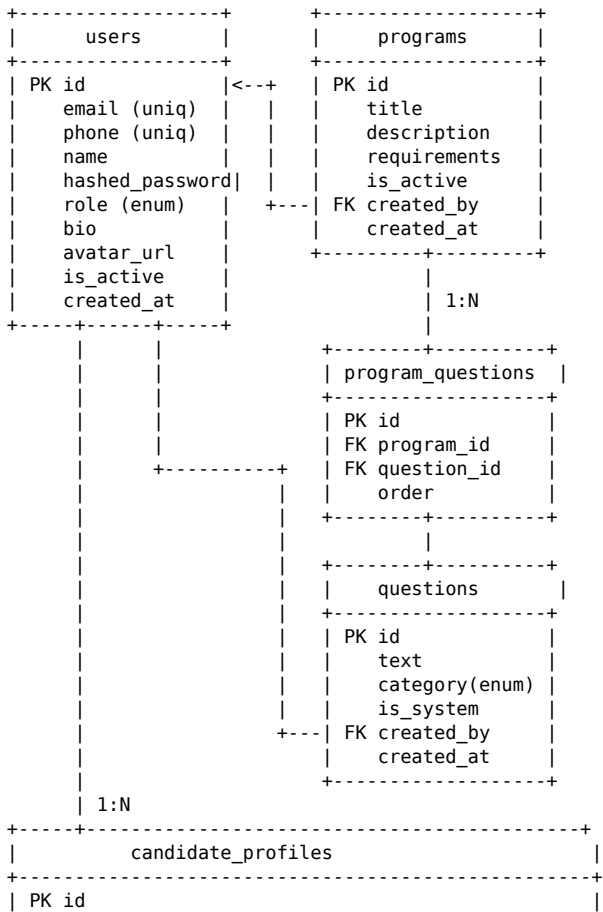
Навигация в header

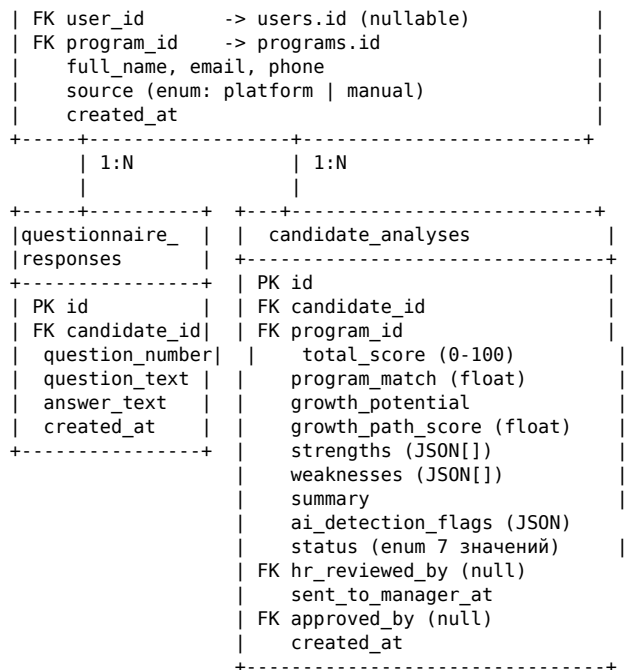
Роль	Пункты меню
Абитуриент	Программы, Мой статус
Координатор отбора	Программы, Отчёты, Зачисленные
Приёмная комиссия	Дашборд

Все роли: клик по имени/аватару -> Профиль, кнопка "Выйти".

Схема базы данных

ER-диаграмма





Таблицы

users

Поле	Тип	Ограничения
id	INTEGER	PK, autoincrement
email	VARCHAR(255)	UNIQUE, INDEX
phone	VARCHAR(50)	UNIQUE, INDEX
name	VARCHAR(255)	
hashed_password	VARCHAR(255)	
role	ENUM	candidate / manager / hr
bio	TEXT	default: ""
avatar_url	VARCHAR(500)	nullable
is_active	BOOLEAN	default: true
created_at	TIMESTAMPTZ	default: now()

programs

Поле	Тип	Ограничения
id	INTEGER	PK
title	VARCHAR(255)	
description	TEXT	
requirements	TEXT	default: ""
is_active	BOOLEAN	default: true
created_by	INTEGER	FK -> users.id
created_at	TIMESTAMPTZ	default: now()

questions (банк вопросов)

Поле	Тип	Ограничения
id	INTEGER	PK
text	TEXT	
category	ENUM	experience / competencies / motivation / potential / leadership / growth_path
is_system	BOOLEAN	default: false
created_by	INTEGER	FK -> users.id, nullable (null для системных)
created_at	TIMESTAMPTZ	default: now()

21 системный вопрос загружается автоматически при старте бекенда (seed).

QuestionCategory enum (6 значений):

Значение	Описание
EXPERIENCE	Опыт и достижения
COMPETENCIES	Навыки и компетенции
MOTIVATION	Мотивация и ценности
POTENTIAL	Потенциал и адаптивность
LEADERSHIP	Лидерские качества
GROWTH_PATH	Траектория роста

program_questions (M2M: программа <-> вопрос)

Поле	Тип	Ограничения
id	INTEGER	PK
program_id	INTEGER	FK -> programs.id
question_id	INTEGER	FK -> questions.id
order	INTEGER	default: 0

candidate_profiles

Поле	Тип	Ограничения
id	INTEGER	PK
user_id	INTEGER	FK -> users.id, nullable
program_id	INTEGER	FK -> programs.id
full_name	VARCHAR(255)	
email	VARCHAR(255)	
phone	VARCHAR(50)	default: ""
source	ENUM	platform / manual
created_at	TIMESTAMPTZ	default: now()

questionnaire_responses

Поле	Тип	Ограничения
id	INTEGER	PK

candidate_id	INTEGER	FK -> candidate_profiles.id
question_number	INTEGER	
question_text	TEXT	
answer_text	TEXT	
created_at	TIMESTAMPTZ	default: now()

candidate_analyses

Поле	Тип	Ограничения
id	INTEGER	PK
candidate_id	INTEGER	FK -> candidate_profiles.id
program_id	INTEGER	FK -> programs.id
total_score	INTEGER	default: 0
program_match	FLOAT	default: 0.0
growth_potential	TEXT	default: ""
growth_path_score	FLOAT	default: 0.0
strengths	JSON	default: []
weaknesses	JSON	default: []
summary	TEXT	default: ""
ai_detection_flags	JSON	default: {}
status	ENUM	7 значений (см. выше)
hr_reviewed_by	INTEGER	FK -> users.id, nullable
sent_to_manager_at	TIMESTAMPTZ	nullable
approved_by	INTEGER	FK -> users.id, nullable
created_at	TIMESTAMPTZ	default: now()

Поле ai_detection_flags (JSON) — результат проверки на AI-генерацию:

```
{
  "is_ai_suspected": true,
  "confidence": 0.82,
  "flags": [
    "sentence_uniformity",
    "low_type_token_ratio",
    "chatgpt_patterns_ru",
    "structured_formatting",
    "excessive_length"
  ],
  "details": {
    "sentence_uniformity": 0.91,
    "type_token_ratio": 0.34,
    "chatgpt_patterns_count": 5,
    "has_structured_formatting": true,
    "avg_answer_length": 487
  }
}
```

5 эвристик AI-детекции:

Эвристика	Описание
sentence_uniformity	Однородность длин предложений (AI пишет равномерно)
low_type_token_ratio	Низкое лексическое разнообразие (TTR < 0.4)
chatgpt_patterns_ru	Паттерны ChatGPT в русском тексте (характерные обороты)

structured_formatting	Чрезмерная структурированность (списки, нумерация)
excessive_length	Избыточная длина ответов (> 300 слов на вопрос)

API-эндпоинты

Все API под префиксом `/api/v1/`.

Auth (/auth)

Метод	Путь	Доступ	Описание
POST	/auth/register	Public	Регистрация (email, phone, name, password) -> JWT
POST	/auth/login	Public	Вход (email, password) -> JWT
GET	/auth/me	JWT	Текущий пользователь

Profile (/profile)

Метод	Путь	Доступ	Описание
GET	/profile/	JWT	Получить свой профиль
PUT	/profile/	JWT	Обновить имя, телефон, bio
POST	/profile/avatar	JWT	Загрузить аватар (multipart, max 5MB)

Programs (/programs)

Метод	Путь	Доступ	Описание
GET	/programs/	Public	Список активных программ
GET	/programs/{id}	Public	Детали программы с вопросами

Candidates (/candidates)

Метод	Путь	Доступ	Описание
POST	/candidates/submit-questionnaire	Candidate	Отправить анкету -> создать profile + answers + AI analysis
GET	/candidates/my-status	Candidate	Статус своей последней заявки

Manager (/manager) — Приёмная комиссия

Метод	Путь	Доступ	Описание
-------	------	--------	----------

GET	/manager/candidates	Manager	Абитуриенты со статусом SENT_TO_MANAGER
GET	/manager/candidates/{id}/analysis	Manager	Полный AI-анализ
POST	/manager/candidates/{id}/approve	Manager	Зачислить -> APPROVED
POST	/manager/candidates/{id}/reject	Manager	Отклонить -> REJECTED

HR (/hr) — Координатор отбора

Метод	Путь	Доступ	Описание
GET	/hr/programs	HR	Все программы
POST	/hr/programs	HR	Создать программу
GET	/hr/programs/{id}	HR	Детали программы с вопросами
GET	/hr/programs/{id}/candidates	HR	Все абитуриенты по программе (все статусы)
GET	/hr/questions/bank	HR	Банк вопросов (системные + кастомные)
POST	/hr/questions	HR	Создать кастомный вопрос
POST	/hr/programs/{id}/questions	HR	Привязать вопросы к программе
GET	/hr/candidates	HR	Абитуриенты со статусом ANALYZED
GET	/hr/candidates/{id}	HR	Полный AI-анализ абитуриента
GET	/hr/candidates/{id}/dossier	HR	Досье: профиль + AI + AI-флаги + ответы
POST	/hr/candidates/{id}/send-to-manager	HR	Отправить в приёмную комиссию
GET	/hr/approved	HR	Зачисленные абитуриенты
POST	/hr/candidates/{id}/invite	HR	Отправить приглашение абитуриенту

Health

Метод	Путь	Доступ	Описание
GET	/health	Public	Backend health check

ML Service (порт 8001):

Метод	Путь	Описание
POST	/api/v1/analyze	AI-анализ ответов абитуриента
POST	/api/v1/detect-ai	Детекция AI-генерированного текста
GET	/health	ML service health check

Аутентификация и авторизация

Механизм

- Пароли: bcrypt (passlib + bcrypt 4.0.1)
- Токены: JWT (python-jose), алгоритм HS256
- Access token: 30 минут
- Refresh token: 7 дней
- Хранение на клиенте: localStorage.access_token

Регистрация

- Роль всегда CANDIDATE (зашиито на бекенде)
- Валидация: уникальность email + уникальность phone
- Координатор отбора и Приёмная комиссия создаются вручную через БД

Поток

Клиент	Backend	PostgreSQL
-- POST /auth/register ----->		
{name, email, phone, pass}	-- check email unique ----->	
	-- check phone unique ----->	
	-- INSERT user (role=cand) ->	
<-- {access_token, refresh} ----		
-- GET /auth/me ----->		
Authorization: Bearer AT	-- decode JWT -> user_id -->	
<-- {id, email, phone, name,	<-- user row -----	
role, bio, avatar_url} ----		
-- Redirect по роли ----->		
candidate -> /programs		
manager -> /manager		
hr -> /hr		

Role-based access (frontend)

- ProtectedRoute компонент проверяет isAuthenticated + allowedRoles
- Неавторизован -> /auth
- Чужая роль -> редирект на дефолтную страницу своей роли

Frontend-архитектура

State Management

- Zustand (authStore) — пользователь, JWT, login/register/logout/hydrate
- Гидратация при загрузке: проверка токена через GET /me

Роутинг

- React Router v6 — вложенные маршруты через +
- RootRedirect — при входе на / редирект по роли

API-клиент

- Axios с interceptors:
- Request: подставляет Authorization: Bearer из localStorage
- Response: при 401 -> очистка токена -> редирект на /auth

Страницы (16)

Страница	Файл	Описание
AuthPage	AuthPage.tsx	Табы Вход/Регистрация
ProfilePage	ProfilePage.tsx	Аватар, имя, телефон, bio
ProgramsPage	ProgramsPage.tsx	Список программ (для абитуриента)
CandidateQuestionnairePage	CandidateQuestionnairePage.tsx	Пошаговая анкета
CandidateStatusPage	CandidateStatusPage.tsx	Статус заявки
ManagerDashboardPage	ManagerDashboardPage.tsx	Абитуриенты SENT_TO_MANAGER
ManagerCandidateDetailPage	ManagerCandidateDetailPage.tsx	AI-анализ + зачислить/отклонить
HrDashboardPage	HrDashboardPage.tsx	Программы + абитуриенты после AI
HrProgramCreatePage	HrProgramCreatePage.tsx	Создание программы + выбор вопросов
HrProgramDetailPage	HrProgramDetailPage.tsx	Программа + вопросы + абитуриенты
HrCandidateReviewPage	HrCandidateReviewPage.tsx	AI-анализ + AI-флаги + отправка в комиссию
HrReportsPage	HrReportsPage.tsx	Отчёты: программы + кол-во абитуриентов
HrReportProgramCandidatesPage	HrReportProgramCandidatesPage.tsx	Абитуриенты по программе (все статусы)
HrReportCandidateAnalysisPage	HrReportCandidateAnalysisPage.tsx	AI-анализ + кнопка "Досье"
HrCandidateDossierPage	HrCandidateDossierPage.tsx	Досье: профиль + AI + AI-флаги + ответы
HrApprovedPage	HrApprovedPage.tsx	Зачисленные -> приглашение

ML-сервис

Текущее состояние

- Отдельный FastAPI на порту 8001
- Эндпоинт POST /api/v1/analyze — AI-анализ ответов абитуриента
- Эндпоинт POST /api/v1/detect-ai — детекция AI-генерированного текста
- AI-анализ может эмулироваться на бекенде (mock scores при submit-questionnaire)

AI-детекция текста (rule-based)

Система детекции AI-генерированных ответов использует 5 эвристик:

- 1. Sentence Uniformity — анализ равномерности длин предложений. AI-модели генерируют предложения похожей длины
- 2. Type-Token Ratio (TTR) — отношение уникальных слов к общему числу. Низкий TTR (< 0.4) характерен для AI
- 3. ChatGPT Patterns (RU) — поиск характерных оборотов ChatGPT в русском тексте (канцеляризмы, шаблонные связки)
- 4. Structured Formatting — детекция избыточной структурированности (маркированные списки, нумерация, подзаголовки)
- 5. Excessive Length — выявление чрезмерно длинных ответов (> 300 слов на вопрос)

Результат сохраняется в поле ai_detection_flags (JSON) таблицы candidate_analyses.

Growth Path Score

Новый показатель growth_path_score (Float, 0.0-1.0) в таблице candidate_analyses. Оценивает траекторию роста абитуриента на основе ответов из категорий GROWTH_PATH и POTENTIAL. Учитывает:

- Наличие чёткого плана развития
- Осознанность выбора направления
- Готовность к долгосрочному обучению

Целевая архитектура

- GPU: NVIDIA RTX 3070 Ti (8GB VRAM)
- Рекомендуемые модели:
 - cointegrated/rubert-tiny2 (~120MB) — быстрый
 - ai-forever/sbert_large_nlu_ru (~1.3GB) — качественный
 - sentence-transformers/paraphrase-multilingual-MiniLM-L12-v2 (~470MB) — баланс
- Pipeline: preprocessing -> embeddings -> scoring -> program matching -> AI detection -> summary generation

Банк системных вопросов (21 шт.)

#	Категория	Вопросы (сокращённо)
1-4	Опыт (EXPERIENCE)	Достижения, проблемы, неудачи, сложные задачи
5-8	Компетенции (COMPETENCIES)	Навыки, обучаемость, командная работа, новые задачи
9-12	Мотивация (MOTIVATION)	Интерес к программе, цели, ценности, инициатива
13-15	Потенциал (POTENTIAL)	Свобода проекта, реакция на изменения, развитие
16-18	Лидерство (LEADERSHIP)	Командное руководство, принятие решений, ответственность
19-21	Траектория роста (GROWTH_PATH)	План развития, выбор направления, долгосрочное видение

Структура проекта

```

inVisionU/
├── docker-compose.yml          # Production compose
├── docker-compose.dev.yml      # Dev override (hot-reload)
├── .env                        # Переменные окружения
├── .gitignore
├── README.txt
├── frontend/                  # React SPA
│   ├── Dockerfile / Dockerfile.dev
│   ├── package.json
│   ├── vite.config.ts
│   ├── tailwind.config.js
│   ├── index.html
│   └── src/
│       ├── main.tsx           # Entry point
│       ├── App.tsx            # Роутинг (22 маршрута)
│       ├── index.css           # Tailwind imports
│       ├── api/client.ts       # Axios + JWT interceptors
│       ├── store/authStore.ts  # Zustand auth state
│       ├── types/index.ts      # TypeScript интерфейсы
│       ├── components/
│       │   └── ProtectedRoute.tsx # Role-based route guard
│       ├── layouts/
│       │   └── MainLayout.tsx   # Header + navigation + Outlet
│       └── pages/              # 16 страниц
│           ├── AuthPage.tsx
│           ├── ProfilePage.tsx
│           ├── ProgramsPage.tsx
│           ├── CandidateQuestionnairePage.tsx
│           ├── CandidateStatusPage.tsx
│           ├── ManagerDashboardPage.tsx
│           ├── ManagerCandidateDetailPage.tsx
│           ├── HrDashboardPage.tsx
│           ├── HrProgramCreatePage.tsx
│           ├── HrProgramDetailPage.tsx
│           ├── HrCandidateReviewPage.tsx
│           ├── HrReportsPage.tsx
│           ├── HrReportProgramCandidatesPage.tsx
│           ├── HrReportCandidateAnalysisPage.tsx
│           ├── HrCandidateDossierPage.tsx
│           └── HrApprovedPage.tsx
├── backend/                   # FastAPI API
│   ├── Dockerfile / Dockerfile.dev
│   ├── requirements.txt
│   └── app/
│       ├── main.py            # FastAPI app + lifespan + seed
│       ├── core/
│       │   ├── config.py       # Pydantic Settings
│       │   ├── database.py     # SQLAlchemy async engine
│       │   └── security.py      # JWT + bcrypt
│       ├── models/             # 7 SQLAlchemy моделей
│       │   ├── user.py
│       │   ├── program.py
│       │   ├── question.py
│       │   ├── program_question.py
│       │   ├── candidate.py
│       │   ├── questionnaire.py
│       │   └── analysis.py
│       ├── schemas/            # Pydantic DTOs
│       │   ├── auth.py
│       │   ├── profile.py
│       │   ├── program.py
│       │   ├── question.py
│       │   └── candidate.py
│       ├── api/v1/              # 6 роутеров, 25+ эндпоинтов
│       │   ├── auth.py
│       │   ├── profile.py
│       │   ├── programs.py
│       │   ├── candidates.py
│       │   ├── manager.py
│       │   └── hr.py
│       ├── services/
│       │   ├── analysis_service.py # HTTP клиент к ML
│       │   └── ai_detection.py    # Rule-based AI детекция
│       └── tasks/               # Celery
│           ├── analysis.py
│           └── notifications.py
├── ml-service/                # ML FastAPI
│   ├── Dockerfile / Dockerfile.dev
│   └── requirements.txt

```

```
|
|
|└─ app/
|   ├── main.py
|   ├── api/
|   │   ├── analyze.py           # POST /api/v1/analyze
|   │   └── detect_ai.py        # POST /api/v1/detect-ai
|   └── services/
|       ├── nlp_analyzer.py      # ML pipeline для анализа
|       └── ai_detector.py       # Rule-based AI детекция
|
|└─ docs/
|   ├── ARCHITECTURE.md          # Документация
|   ├── FRONTEND_SPEC.md        # Этот документ
|   ├── BACKEND_SPEC.md
|   ├── ML_SPEC.md
|   └── API_SPEC.md
```